**Mathematical Modeling and Algorithm Application**
ISSN: 3006-0842 | Vol. 3, No. 2, 2024

# YOLO V8 Network Lightweight Method Based on Pruning and Quantization

**Yiyang Cheng***

Sydney Smart Technology College, Northeastern University at Qinhuangdao, Qinhuangdao, 050011, China
* **Corresponding author**: 202219182@stu.neu.edu.cn

**Abstract:** With the development of deep learning, convolutional neural networks have been widely used as core algorithms in fields such as computer vision, natural language processing, and speech processing, surpassing the performance of traditional algorithms. However, its complex structure and massive computational load make many algorithms dependent on GPU implementation, which limits its application on mobile devices with limited resources and high real-time requirements. This article explores in detail the lightweighting process of the YOLO v8 model, aiming to optimize its performance in resource constrained environments by combining BN layer scale factor pruning and int8 quantization techniques under the TensorRT framework. Firstly, using the scale factor in the BN layer as an evaluation criterion, redundant channels in the network are identified, effectively reducing the number of model parameters and computational requirements; Then deploy the model in NVIDIA's TensorRT framework for int8 quantization. This process compresses the floating-point operations of the model into low bit int8 operations, significantly reducing computational complexity and memory usage; And validated through COCO dataset experiments during the process. The results show that YOLO v8, after pruning and quantization, can achieve a compression ratio of 64.2% with a loss detection accuracy of about 4%. This study provides a comprehensive solution for the efficient deployment of deep learning models, especially in application scenarios that require balancing performance and resource efficiency, which has important practical significance. Future work will focus on further optimizing pruning and quantization strategies to adapt to a wider range of network architectures and application requirements.

**Keywords:** YOLO v8; Model pruning; Weight quantization; Object detection.

## 1. Introduction

With the rapid development of deep learning technology, Convolutional Neural Network (CNN), as one of the important algorithms in deep learning, has achieved outstanding results in computer vision fields such as image recognition and object detection, even surpassing human performance in certain tasks. Among them, YOLO (You Only Look Once) is an excellent real-time object detection model that has gained widespread attention due to its efficient architecture and excellent performance. Especially in the field of transportation, target recognition based on YOLO convolutional neural network is of great significance in the fields of transportation, criminal investigation, as well as bringing economic and social benefits.

The YOLO series model adopts a single forward propagation approach for object detection, which has higher inference speed compared to traditional two-stage object detection algorithms such as the R-CNN series, and can meet the needs of real-time applications. YOLO v8 is the cutting-edge version of the YOLO series, which adopts a deeper convolutional neural network architecture, including twenty-three layers (YOLO v8n) of convolutional layers, pooling layers, and fully connected layers. This complex network structure greatly improves the performance of YOLO v8 in object detection accuracy and coverage, but at the same time, it also leads to a geometric increase in the model's parameter and computational complexity. The complete model has a size of several hundred MB, which makes it difficult to deploy and run on mobile devices with limited storage capacity and computing power, limiting its promotion in practical applications. Therefore, how to effectively compress the model size and reduce computational complexity while maintaining the performance of YOLO v8 model has become an important problem that urgently needs to be solved.

Therefore, this paper proposes a compression method of YOLO v8 model based on weight pruning and quantitative training. Firstly, redundant channels are removed by pruning structured channels and scale factor γ of BN layer. Then, the neural network is quantized, and the floating-point weight of the model is quantized by int8 according to max-min to further reduce the storage requirements and computational complexity of the model. This method combining channel pruning and quantization is expected to significantly reduce the model size while maintaining the performance of the YOLO v8 model, making it more suitable for deployment on resource constrained mobile devices.

The experimental results show that the proposed method can effectively compress the size of the YOLO v8 model with a small loss of accuracy, which provides a feasible solution for the application of the model in the mobile terminal. This can not only promote the application of deep learning technology in mobile terminals, but also provide new ideas and methods for research in related fields.

## 2. Literature review

In order to improve the performance of convolutional neural network, researchers continue to try innovative design of neural network structure. By combining convolution kernels of different sizes in the same layer, the concept module introduced by GoogleNet significantly improves the width and expression ability of the network, while maintaining the controllability of the computational cost [1]. After that, ResNet successfully solved the problem of gradient disappearance in deep network training by

introducing the residual module, so that the number of network layers can be increased to a depth not previously realized [2]. In addition, DenseNet greatly improves the efficiency of information flow and gradient propagation by connecting the characteristic graphs between all layers, thus further improving the performance of the model [3]. These structural optimization methods greatly enhance the performance of neural network, but also bring the problems of too large parameters, over fitting phenomenon, and the difficulty of deployment in embedded devices. To solve these problems, researchers have proposed many lightweight convolutional neural network methods. In addition to proposing compact network models such as MobileNet, SqueezeNet, ShuffleNet, and DenseNet, pruning and quantization methods have become the mainstream algorithms of neural network lightweight.

Pruning algorithm aims to improve the computational efficiency and storage adaptability of the network by removing redundant parameters. The new research direction not only focuses on static pruning methods, but also develops dynamic pruning strategies, which can adaptively adjust the network structure according to the characteristics of the input data, so as to achieve higher compression rate without significantly reducing the accuracy [4]. In addition, pruning methods based on neural structure search (NAS) are also gradually emerging. By searching for the best pruning strategy, the network structure is automatically optimized, making the pruning process more efficient and intelligent [5]. In addition, by combining knowledge distillation technology, the knowledge of large model is transferred to the light model after pruning, so as to make up for the performance loss caused by pruning [6]. These improvements not only improve the deployment ability of CNN on resource constrained devices, but also provide new ideas and methods for wider applications.

In recent years, the quantization algorithm of neural network has made significant progress, which is mainly divided into the following three categories: unified quantization of weight and activation, asymmetric quantization and mixed precision quantization. In the unified quantization method, learned step size quantization (LSQ) significantly improves the flexibility and accuracy of quantization by dynamically adjusting the quantization step size during the training process. However, it still has some limitations in dealing with the complexity of different levels, and it is difficult to adapt to the needs of all network layers [7]. Therefore, parameterized clipping activation (PACT) further optimizes the quantization process by introducing a parameterized clipping function, so that different layers can adjust the quantization strategy adaptively, so as to achieve a better balance between accuracy and efficiency [8]. The asymmetric quantization method allows different layers to use different quantization parameters, a method proposed improves the performance in terms of flexibility, but due to the difference of quantization parameters, it may lead to additional computational overhead [9]. In response, a dynamic fixed point quantized training approach (AdaPT) is proposed to further reduce the loss of accuracy and computational complexity by dynamically adjusting the quantization bit width of each layer [10]. Hybrid precision quantization achieves the best balance between performance and efficiency by applying different precision quantization strategies in different network parts. Hardware-aware automated quantization (HAQ) determines the optimal quantization accuracy of each layer through automated search, which significantly improves the execution efficiency of the network, but its search process may introduce high computational costs [11]. To improve this, a hybrid precision strategy based on reinforcement learning (AutoQ) can reduce the consumption of computing resources while maintaining high precision, and further improve the effectiveness and adaptability of quantization [12].

# 3. Methodology

## 3.1. Pruning algorithm based on BN layer scaling factor $\gamma$

### 3.1.1. *Sparsity training.*

In recent years, many parameter importance evaluation methods can obtain approximate channel importance ranking, but without sparsity training, the channel weights may be evenly distributed, making subsequent pruning mistakenly delete important channels and degrade the performance of the model. After sparsity training, the network has adapted to the state that some parameters are zero, which can minimize the loss of information and improve the pruning efficiency. YOLO v8 network adds BN layer to speed up the training process. The function of BN layer is to keep the same distribution of the activation input value of each layer during training, to avoid the internal covariate offset problem, so that the activation input value is always in the sensitive region, away from the derivative saturation region, and to solve the gradient disappearance problem of back propagation. The BN layer normalizes the data by optimizing the scale factor $\gamma$ and the offset coefficient $\beta$, so that the network can learn the characteristic distribution of the output. The BN layer is implemented by using $z_{in}$ and $z_{out}$ as the input and output of the BN layer. $\beta$ represents the current minimum block. The BN layer performs the following conversion:

$$z = \frac{z_{in} - \mu_\beta}{\sqrt{\sigma_\beta{}^2 + \varepsilon}} \tag{1}$$

$$z_{out} = \gamma z + \beta \tag{2}$$

Where $\mu_\beta$ and $\sigma^2$ are the mean and variance of the input values in the current layer respectively, that is:

$$\mu_\beta = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{3}$$

$$\sigma^2 = \frac{1}{m}\sum_{i=1}^{m} x_i - \mu_\beta \tag{4}$$

Sparsity training is aimed at the scale factor $\gamma$ and the offset coefficient $\beta$. Add sparsity constraints on both in the loss term. The larger the $\lambda$ coefficient, the more severe the sparse constraints jump

$$L = \sum_{(x,y)} l(f(x), y) + \lambda_1 \sum_\gamma g(\gamma) + \lambda_2 \sum_\beta g(\beta) \tag{5}$$

For $L_1$ sparse constraint, there are:

$$g(\gamma) = |\gamma|, g(\beta) = |\beta| \tag{6}$$

Compared with the original gradient term, the scale factor and bias term of BN will increase the following gradients:

$$\Delta\gamma = \frac{\partial(\lambda_1 * g(\gamma))}{\partial\gamma} = \lambda_1 * \text{sign}(\gamma) \tag{7}$$

$$\Delta\beta = \frac{\partial(\lambda_2 * g(\beta))}{\partial\beta} = \lambda_2 * \text{sign}(\beta) \tag{8}$$

During the training process, the $\lambda_1$ parameter is gradually reduced to reduce the constraint on $\gamma$

$$\lambda_1 = 0.01 * \left(1 - 0.9 * \frac{e}{ne}\right) \tag{9}$$

The sparsity model is obtained by training the network sparsity. At this time, the importance of the channel is evaluated, and the pruning algorithm is used for pruning.

### 3.1.2. *Pruning Algorithm.*

The $\gamma$ scaling factor in BN layer has a weight corresponding to the characteristic graph in each convolution layer, which can be used as the evaluation coefficient of the current layer's contribution to the accuracy of the whole model. The greater the $\gamma$ value, the greater the contribution of the current layer to the accuracy of the model, and vice versa. When the weight $\gamma$ corresponding to the input channel appears $\gamma = 0$ or $\gamma \approx 0$, it can be considered that the contribution of the channel to the overall accuracy of the model can be ignored. The convolution kernel corresponding to the corresponding upper and lower convolution layers is cut off to compress the volume of the model and reduce the amount of calculation.

The structure of YOLO v8 is as follows:

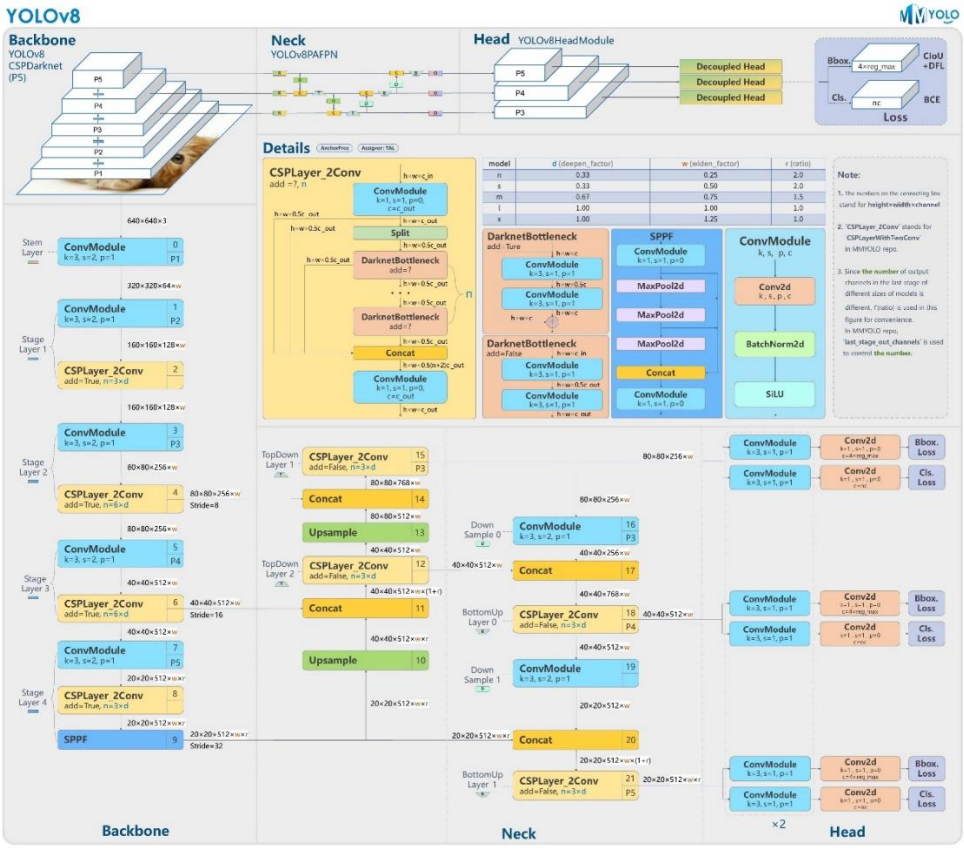



**Figure 1.** YOLO v8 network structure

Each conv layer in the network structure contains a BN layer. When pruning BN channels, on the one hand, the number of output channels and the corresponding weight of conv need to be cut off, on the other hand, the number and weight of input channels of conv in the next layer need to be cut off.

The number of 0, 1 and 2 channels in the first three layers is small and each channel is important for feature extraction, so pruning is not required; The outputs of layers 4, 6 and 9 involve the splicing of channels in the head layer, and the structure is complex, so they are not pruned; In addition, other conv discontinuous parts, such as the split operation between the conv layer in C2F and the bottleneck, and the Upsample, Concat and other operations are interspersed between C2F in FPN. These parts are also inconvenient to prune.

Therefore, the pruning part includes:

**a. Between modules**

Backbone:

$$Conv(3) \Rightarrow C2f(4)$$
$$Conv(5) \Rightarrow C2f(6)$$
$$Conv(7) \Rightarrow C2f(8)$$
$$C2f(8) \Rightarrow SPPF(9)$$

Head:

$$C2f(15) \Rightarrow [Conv(16), Conv(Detect.cv2[0][0]), Conv(Detect.cv3[0][0])]$$
$$C2f(18) \Rightarrow [Conv(19), Conv(Detect.cv2[1][0]), Conv(Detect.cv3[1][0])]$$
$$C2f(21) \Rightarrow [Conv(Detect.cv2[2]), Conv(Detect.cv3[2])]$$

**b. In module**

Bottleneck in C2f:

$$Conv(Bottleneck.cv1) \Rightarrow Conv(Bottleneck.cv2)$$

cv2, cv3 in Detect:

$$Conv(Detect.cv2[0][0]) \Rightarrow Conv(Detect.cv2[0][1])$$
$$Conv(Detect.cv2[0][1]) \Rightarrow Conv2d(Detect.cv2[0][2])$$
$$Conv(Detect.cv3[0][0]) \Rightarrow Conv(Detect.cv3[0][1])$$
$$Conv(Detect.cv3[0][1]) \Rightarrow Conv2d(Detect.cv3[0][2])$$
$$Conv(Detect.cv2[1][0]) \Rightarrow Conv(Detect.cv2[1][1])$$

$$Conv(Detect.cv2[1][1]) => Conv2d(Detect.cv2[1][2])$$
$$Conv(Detect.cv3[1][0]) => Conv(Detect.cv3[1][1])$$
$$Conv(Detect.cv3[1][1]) => Conv2d(Detect.cv3[1][2])$$
$$Conv(Detect.cv2[2][0]) => Conv(Detect.cv2[2][1])$$
$$Conv(Detect.cv2[2][1]) => Conv2d(Detect.cv2[2][2])$$
$$Conv(Detect.cv3[2][0]) => Conv(Detect.cv3[2][1])$$
$$Conv(Detect.cv3[2][1]) => Conv2d(Detect.cv3[2][2])$$

## 3.2. Quantization algorithm

The $\gamma$ value on which the above pruning is based may have a low pruning rate due to high accuracy requirements, so the remaining weight parameters need to be quantized. The parameters used by YOLO v8 for image recognition are 32-bit floating-point numbers, but the calculation cost of floating-point numbers is greater than that of integer data, and its quantization can significantly compress the remaining weight parameters and reduce the calculation cost. The realization of quantization in neural network requires that convolution, matrix multiplication, activation function, pooling, splicing and other operations be converted into equivalent 8-bit integer operations, and then quantization and inverse quantization operations are added before and after the operation to convert the input from floating-point number to 8-bit integer, and inverse quantization operation converts the output from 8-bit integer to floating-point number.

The specific method of int8 quantization is to convert the input float32 type data to int8 type, and then transfer it to the convolution layer. At the same time, the weight in the convolution layer is also converted to int8 type. The quantization operation needs to convert a set of floating-point number inputs to uint8 (0~255) types. First, calculate the minimum and maximum values of this set of inputs, and then use the following formula to quantize and inverse quantize each input data.

$$q = \frac{x-min}{max-min} \times 255 \tag{10}$$

$$x = \frac{q \times (max-min)}{255} + min \tag{11}$$

The formula for quantizing float32 data type into fixed-point data type is as follows:

$$x_q = \left[\frac{x_f}{s}\right] - \phi_x \tag{12}$$

$$s = \frac{r_{f_{max}} - r_{f_{min}}}{2^n - 1} \tag{13}$$

$$\phi_x = \left[\frac{x_{f_{min}}}{s}\right] \tag{14}$$

Where $s$ is the quantization interval, $n$ is the number of quantized bits, and $\phi_x$ is the quantization value corresponding to 0. $r_{f_{max}}$ and $r_{f_{min}}$ are the maximum and minimum values of float data. In the framework of TensorRT,[x]represents the rounding function:

$$[x] = \text{sgn}(x) * \lfloor |x| + 0.5 \rfloor \tag{15}$$

Where $\text{sgn}(x)$ is a symbolic function and ⌊ x ⌋ represents rounding down.

Convolution calculation is designed for convolution neural network. After the convolution neural network is quantized, the calculation principle of convolution operation is as follows:

$$a_f = \sum(x_f * \omega_f) \tag{16}$$

$$a_f = s_x s_\omega \sum(x_q + \phi_x) * (\omega_q + \phi_\omega) \tag{17}$$

At the same time:

$$a_f = s_a(a_q + \phi_a) \tag{18}$$

It can be concluded that:

$$a_q = \left[\frac{s_x s_\omega}{s_a}\sum(s_q + \phi_x) * (\omega_q + \phi_\omega)\right] - \phi_a \tag{19}$$

Where $x_q$, $\omega_q$ and $a_q$ are quantized inputs, weights and convolution outputs respectively; $s_x$, $s_\omega$ and $s_a$ are the scale factors of input, weight and convolution output respectively; $\phi_x$, $\phi_\omega$ and $\phi_a$ are the zeros of input, weight and convolution output respectively.

In this paper, the lightweight method of YOLO v8, which combines pruning and parameter quantization, is proposed as follows: firstly, the model is sparsely trained through the training set, and then the whole network structure is pruned through the BN layer parameters and the weights in the convolution layer and the full connection layer of the whole network structure, and then the remaining parameters are quantized after pruning. The specific process is as follows: ① set up the network model training set; ② The corresponding network model is trained for sparsity; ③ Prune the converged network model based on BN layer parameters and weight parameters based on convolution layer and full connection layer in the whole network structure; ④ Quantize the remaining parameters after pruning; ⑤ Fine tune training to restore accuracy; ⑥ The final model outputs relevant data and detects the accuracy. The specific process is shown in Figure 2 and Figure 3.

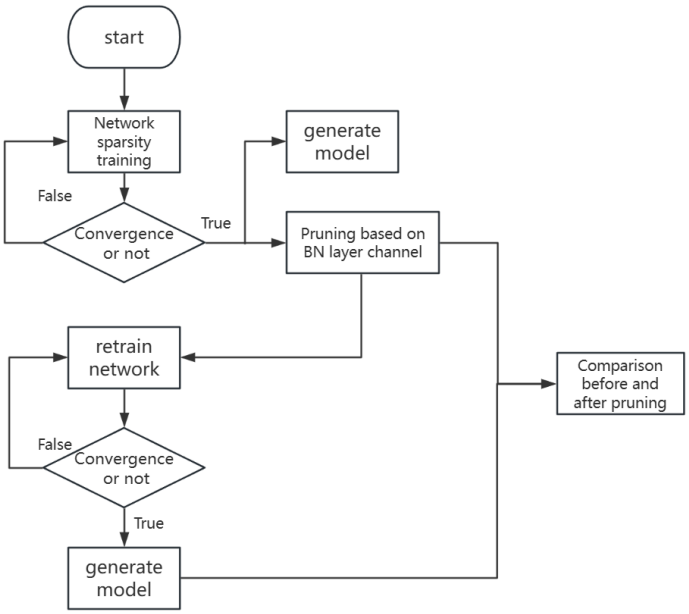



**Figure 2.** Pruning process

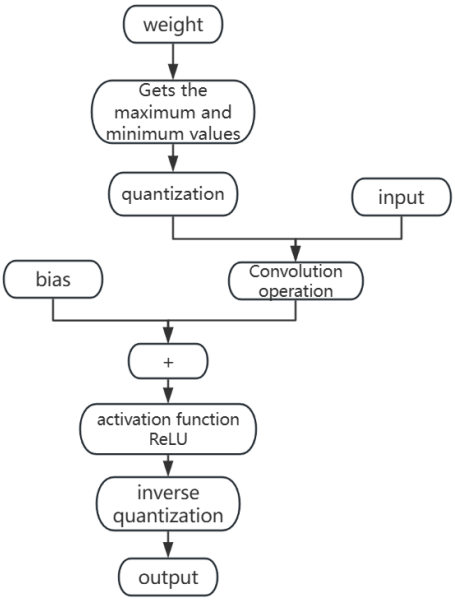



**Figure 3.** Quantization process

# 4. Results

## 4.1. Analysis of pruning results

This section verifies the effectiveness of channel pruning algorithm based on BN layer on YOLO v8n network. In this experiment, graphics card acceleration is used. The GPU is NVIDIA GeForce RTX 3050, and the data set used is coco data set. It can be seen from table 1 that after pruning, the weight of YOLO v8n network is reduced from about $3.1 \times 10^6$ to about $2.5 \times 10^6$, the pruning rate is about 19.3%, the floating-point operands are reduced from about 8.7 GFLOPs to 5.8 GFLOPs, and the model accuracy after pruning can reach 72.2%, which is about 5% lower than the baseline. At the same time, the accuracy and confidence curves of the retraining before and after pruning were compared, and the accuracy decreased in an acceptable range after pruning. Therefore, the feasibility of pruning algorithm can be seen from the experimental results.

**Table 1.** Accuracy of Yolo v8n network before and after pruning.

| Before pruning | | | After pruning | | | Retrain |
|---|---|---|---|---|---|---|
| GFLOPs | Params | Acc | GFLOPs | Params | Acc | |
| 8.7 | 3,151,904 | 0.771 | 5.8 | 2,536,321 | 0.722 | 0.754 |

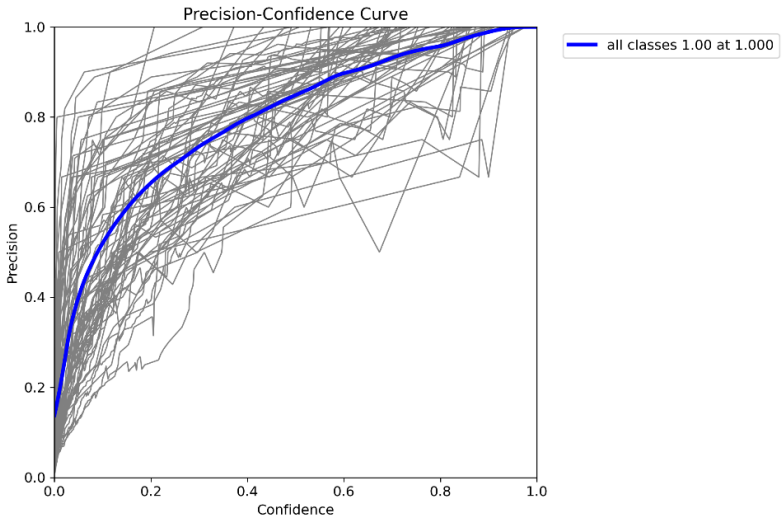



**Figure 4.** Accuracy confidence curve of Yolo v8n network before pruning

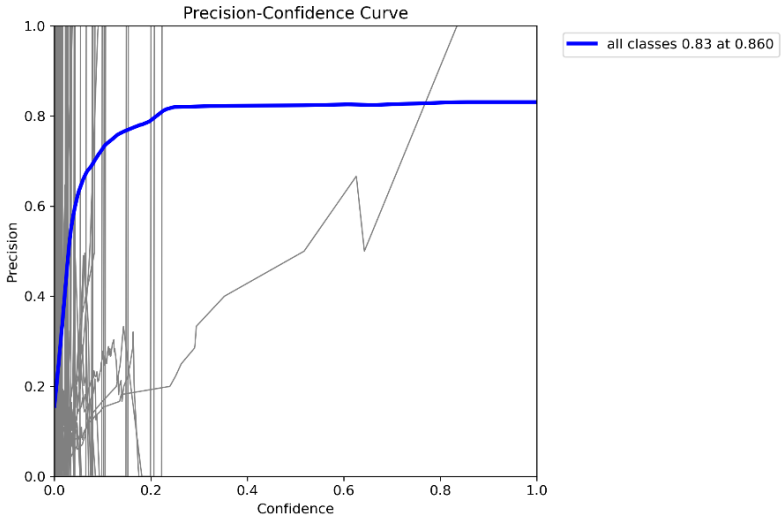



**Figure 5.** Accuracy confidence curve of Yolo v8n network after pruning

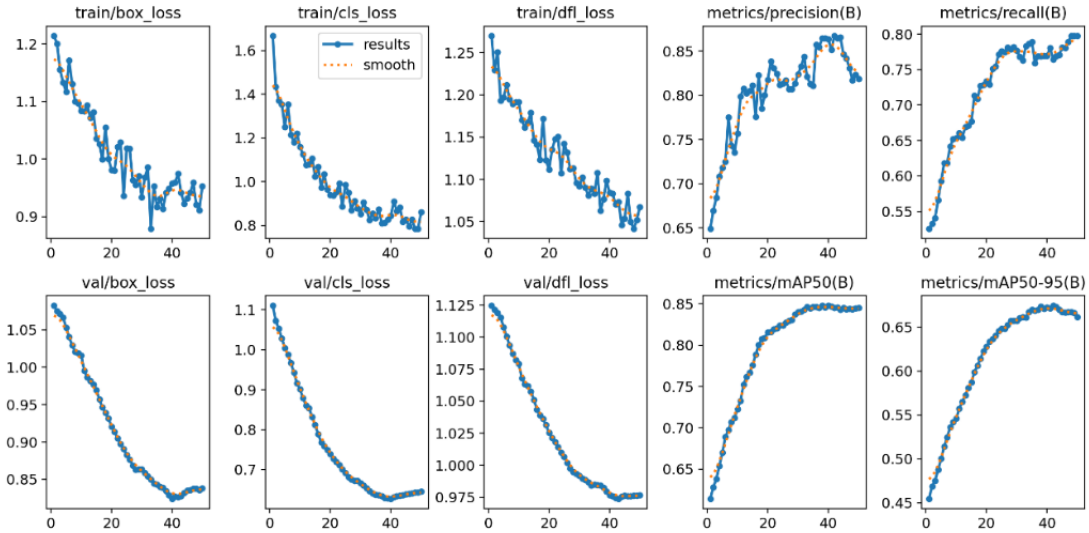



**Figure 6.** Training results of Yolo v8n network before pruning

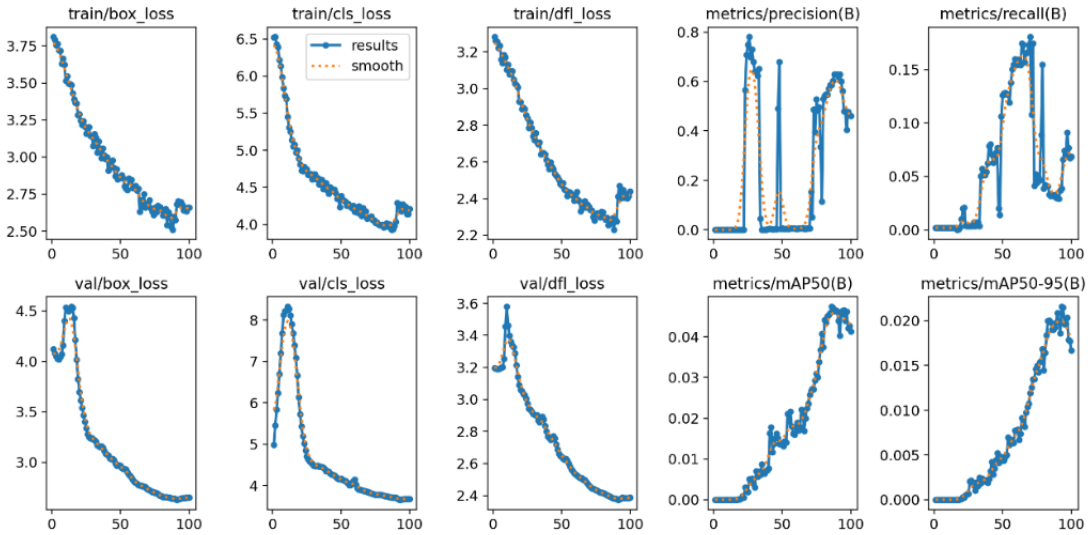



**Figure 7.** Training results of Yolo v8n network after pruning

## 4.2. Int8 quantization of network model under tensorrt framework

After pruning, the weight data is quantized, and the weight with the original data type of float32 is quantized as int8 type. After model calculation, it is restored to float32 type data. The accuracy of the obtained network model is equivalent to that of the original model, but the speed of target detection using the Yolo V8 official bus sample image is higher than that of the original model. The quantization results are shown in Table 2, where speed is the time taken for target detection with the sample image.

**Table 2.** Accuracy of Yolo v8n network before and after quantization.

| Before quantization | | | After quantization | | |
|---|---|---|---|---|---|
| Engine file size | Speed | Acc | Engine file size | Speed | Acc |
| 20.1MB | 53.5ms | 0.754 | 7.2MB | 205ms | 0.722 |

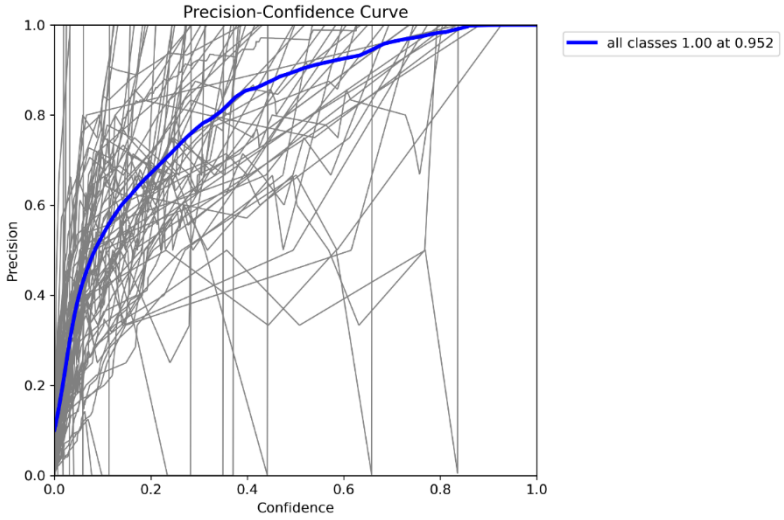



**Figure 8.** Accuracy confidence curve of Yolo v8n network after quantization

According to the network training results, compared with the model after pruning alone before quantization, the compression ratio of the model can reach 64.2%, and the accuracy rate can be reduced by 4.2%. The network can get good compression while the accuracy is not much reduced. Combined with the above experimental data, it can be concluded that the lightweight convolutional neural network Yolo V8 based on pruning quantization is of great significance for deploying to mobile terminals with limited resources.

# 5. Conclusion

In this paper, a lightweight method combining pruning and quantization for convolutional neural network Yolo V8 is proposed. This method mainly discriminates the weights of convolutional neural network after training, reduces the neural network parameters and model volume by pruning the weights and corresponding channels, and then re trains to ensure the accuracy of neural network. After the weights and channels are trimmed, the remaining weights are quantified to further reduce the amount of calculation, and then the model is fine tuned. Finally, the model size is much smaller than the

original model, but the calculation efficiency of the model is much higher than the original model in the case of small decline in model accuracy. The pruning method is based on the scaling factor of BN layer, while the quantization is to int8 quantify all the parameters after pruning, so as to reduce the computational resources. However, this method still has some shortcomings: the pruned channels can be further judged and confirmed, or combined with other parameter importance evaluation methods to avoid pruning important channels; TensorRT has strong hardware support and compatibility in int8 quantization, but its flexibility may be insufficient. For example, specific adjustments and calibrations of the model are required to achieve the best performance on different hardware, which may increase the complexity of development and deployment; When pruning and quantization are used to fine tune large-scale data sets, the cost of computing resources and time is high, which needs to be further studied.

# References


[1] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S.E., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2014). Going deeper with convolutions. 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1-9.

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep Residual Learning for Image Recognition. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.

[3] Huang, G., Liu, Z., & Weinberger, K.Q. (2016). Densely Connected Convolutional Networks. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2261-2269.

[4] Yang, T., Chen, Y., & Sze, V. (2016). Designing Energy-Efficient Convolutional Neural Networks Using Energy-Aware Pruning. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 6071-6079.

[5] Liu, Z., Sun, M., Zhou, T., Huang, G., & Darrell, T. (2018). Rethinking the Value of Network Pruning. ArXiv, abs/1810.05270.

[6] Lan, X., Zhu, X., & Gong, S. (2018). Knowledge Distillation by On-the-Fly Native Ensemble. ArXiv, abs/1806.04606.

[7] Esser, S. K., McKinstry, J. L., Bablani, D., Appuswamy, R., & Modha, D. S. (2020). Learned step size quantization. International Conference on Learning Representations.

[8] Choi, J., Wang, Z., Venkataramani, S., Chuang, P.I., Srinivasan, V., & Gopalakrishnan, K. (2018). PACT: Parameterized Clipping Activation for Quantized Neural Networks. ArXiv, abs/1805.06085.

[9] Banner, R., Nahshan, Y., & Soudry, D. (2019). Post training 4-bit quantization of convolutional networks for rapid-deployment. In Proceedings of the 33rd International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc.

[10] Kummer, L., Sidak, K., Reichmann, T., & Gansterer, W. N. (2023). Adaptive precision training (AdaPT): A dynamic quantized training approach for DNNs. In 2023 SIAM International Conference on Data Mining (SDM), 559-567. https://doi.org/10.1137/1.9781611977653.ch63

[11] Wang, K., Liu, Z., Lin, Y., Lin, J., & Han, S. (2019). HAQ: Hardware-Aware Automated Quantization With Mixed Precision. 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 8604–8612. doi:10.1109/CVPR.2019.00881.

[12] Yang, T., Chen, Y., & Sze, V. (2016). Designing Energy-Efficient Convolutional Neural Networks Using Energy-Aware Pruning. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 6071-6079.